# Software Requirements Specification (SRS)

## Chess Learning Platform (Chess.com–Inspired)

---

# 1. Introduction

This document follows the **IEEE 29148:2018 Software Requirements Specification (SRS)** standard, defining system requirements, constraints, risks, and lifecycle considerations.

### 1.1 Purpose

This Software Requirements Specification (SRS) document defines the functional and non-functional requirements for the development of an online chess learning platform similar to Chess.com. The platform will provide free and paid chess courses, subscription-based learning, interactive lessons powered by the Stockfish chess engine, and analytics-driven progress tracking.

This document serves as a reference for developers, designers, stakeholders, and future maintainers of the system.

---

### 1.2 Scope

The system will: - Allow students to learn chess via structured courses - Offer free and premium (paid) content - Provide subscription-based access - Integrate Stockfish for chess analysis and learning - Track user progress and performance - Support instructors and administrators for content management

The platform will be developed using Django (backend) with optional React-based frontend enhancements.

---

### 1.3 Intended Audience

- Software Developers
- Project Managers
- UI/UX Designers
- Chess Instructors
- Quality Assurance Engineers

---

### 1.4 Definitions, Acronyms, and Abbreviations

- **MVC/MTV**: Model View Controller / Model Template View
- **LMS**: Learning Management System
- **API**: Application Programming Interface

- **SRS**: Software Requirements Specification
- **FEN**: Forsyth–Edwards Notation

---

# 2. Overall Description

## 2.1 Product Perspective

The system is a web-based application built on Django, following a modular app-based architecture. The platform will initially use Django Templates (MTV) and later integrate React for enhanced interactivity.

The system integrates third-party services such as: - Stripe (payments) - Stockfish (chess engine) - Video hosting platforms (Vimeo / YouTube)

---

## 2.2 Product Functions

High-level features include: - User authentication and role management - Course catalog and lesson delivery - Video-based and interactive chess lessons - Subscription and payment management - Chess engine analysis - Progress tracking and analytics - Admin and instructor dashboards

---

## 2.3 User Classes and Characteristics

| User Type | Description |
|---|---|
| Student | Learns chess, enrolls in courses, views progress |
| Instructor | Creates and manages courses and lessons |
| Admin | Manages users, payments, analytics, and system settings |

---

## 2.4 Operating Environment

- Backend: Django + Django REST Framework
- Frontend: Django Templates (initial), React (later)
- Database: PostgreSQL
- Hosting: AWS / DigitalOcean
- Browser Support: Chrome, Firefox, Edge, Safari

---

## 2.5 Design and Implementation Constraints

- Must comply with Stripe payment policies
- Secure handling of user data

- Scalable architecture for future mobile apps
- Chess engine integration must not overload server

---

### 2.6 Assumptions and Dependencies

- Users have stable internet access
- Chess engine computations run server-side
- Video content hosted externally

---

## 3. System Features and Requirements

### 3.1 User Authentication Module

#### Functional Requirements

- User registration and login
- Email verification
- Password reset
- Role-based access control

---

### 3.2 Course Management Module

#### Functional Requirements

- Admin/Instructor can create, edit, delete courses
- Courses categorized by difficulty and topic
- Lessons organized in sequence
- Support for free and paid courses

---

### 3.3 Lesson Delivery Module

#### Functional Requirements

- Video lessons playback
- Text-based lesson material
- Lesson completion tracking
- Resume last watched lesson

---

### 3.4 Enrollment and Progress Tracking Module

**Functional Requirements**

- Students can enroll in courses
- Track lesson completion
- Show course progress percentage
- Store timestamps and activity logs

---

### 3.5 Payment and Subscription Module

**Functional Requirements**

- Integration with Stripe
- One-time payments for courses
- Monthly subscription plans
- Automatic subscription renewal
- Invoice generation

---

### 3.6 Chess Engine Integration Module

**Functional Requirements**

- Load chessboard using FEN/PGN
- Analyze positions using Stockfish
- Display best move suggestions
- Provide evaluation scores
- Detect blunders and inaccuracies

---

### 3.7 Interactive Learning Module

**Functional Requirements**

- Guess-the-move exercises
- Engine-based hints
- Adaptive difficulty
- Feedback after each move

---

### 3.8 Dashboard and Analytics Module

**Functional Requirements**

- Student progress dashboard

- Strength/weakness analysis
- Admin analytics dashboard
- Enrollment and revenue reports

---

### 3.9 Blog and SEO Module

**Functional Requirements**

- Admin can publish blog posts
- SEO-friendly URLs
- Categorized articles

---

## 4. External Interface Requirements

### 4.1 User Interfaces

- Responsive web UI
- Chessboard UI
- Admin dashboard

---

### 4.2 Hardware Interfaces

- No special hardware required

---

### 4.3 Software Interfaces

- Stripe API
- Stockfish engine
- Email services (SMTP)

---

### 4.4 Communication Interfaces

- HTTPS-based API communication

---

## 5. Non-Functional Requirements

### 5.1 Performance Requirements

- Engine analysis response < 2 seconds

- Page load time < 3 seconds

---

### 5.2 Security Requirements

- Encrypted passwords
- Secure payment processing
- Role-based authorization
- CSRF and XSS protection

---

### 5.3 Scalability Requirements

- Support increasing users
- Modular app design

---

### 5.4 Reliability Requirements

- 99% uptime
- Regular backups

---

### 5.5 Usability Requirements

- Beginner-friendly UI
- Clear navigation
- Mobile-responsive design

---

## 6. Project Timeline, Risk Analysis, and Milestones

The project is planned over a duration of **6 months**, divided into structured phases. Each phase includes objectives, risks, mitigation strategies, and associated use cases.

---

### 6.1 Phase 1 – Foundation & Architecture (Month 1)

**Purpose:** Establish a strong technical and architectural base.

**Key Goals:** - Finalize system architecture - Set up Django project structure - Implement authentication and roles

**Deliverables:** - Django project skeleton - Custom user model - Secure login/signup

**Risks & Mitigation:** | Risk | Impact | Mitigation | |-----|-------|-----------| | Poor architecture decisions | High | Design reviews and modular app structure | | Security misconfiguration | High | Follow Django security best practices |

**Primary Use Cases:** - UC-01: User Registration - UC-02: User Login - UC-03: Role Assignment

---

### 6.2 Phase 2 – Course Management & LMS Core (Month 2)

**Purpose:** Enable structured learning via courses and lessons.

**Key Goals:** - Develop course catalog - Implement lesson sequencing - Track progress

**Deliverables:** - Course & lesson modules - Progress tracking

**Risks & Mitigation:** | Risk | Impact | Mitigation | |-----|-------|-----------| | Poor content organization | Medium | Use hierarchical course models | | Scalability issues | Medium | Normalize database schema |

**Primary Use Cases:** - UC-04: Browse Courses - UC-05: Enroll in Free Course - UC-06: View Lesson Content

---

### 6.3 Phase 3 – Payments & Subscriptions (Month 3)

**Purpose:** Introduce monetization.

**Key Goals:** - Stripe integration - Subscription lifecycle handling

**Deliverables:** - Payment module - Subscription management

**Risks & Mitigation:** | Risk | Impact | Mitigation | |-----|-------|-----------| | Payment failures | High | Stripe webhooks and retries | | Unauthorized access | High | Subscription middleware checks |

**Primary Use Cases:** - UC-07: Purchase Course - UC-08: Subscribe to Plan - UC-09: Cancel Subscription

---

### 6.4 Phase 4 – Chess Engine & Interactive Learning (Month 4)

**Purpose:** Add intelligence-driven learning.

**Key Goals:** - Integrate Stockfish - Enable interactive board lessons

**Deliverables:** - Engine analysis API - Interactive chess lessons

**Risks & Mitigation:** | Risk | Impact | Mitigation | |-----|-------|-----------| | High server load | High | Engine depth limits and caching | | Incorrect analysis | Medium | Validate with test positions |

**Primary Use Cases:** - UC-10: Analyze Position - UC-11: Receive Move Feedback

---

### 6.5 Phase 5 – Advanced UI & Analytics (Month 5)

**Purpose:** Improve UX and insights.

**Key Goals:** - Introduce REST APIs - Add React dashboard

**Deliverables:** - Analytics dashboard - React-based UI

**Risks & Mitigation:** | Risk | Impact | Mitigation | |-----|-------|----------| | API inconsistency | Medium | API documentation and testing | | UX complexity | Medium | Iterative UI testing |

**Primary Use Cases:** - UC-12: View Progress Analytics - UC-13: Receive Notifications

---

### 6.6 Phase 6 – Testing, Deployment & Launch (Month 6)

**Purpose:** Production readiness.

**Key Goals:** - System testing - Deployment

**Deliverables:** - Live system - Deployment documentation

**Risks & Mitigation:** | Risk | Impact | Mitigation | |-----|-------|----------| | Deployment failure | High | Staging environment testing | | Data loss | High | Automated backups |

**Primary Use Cases:** - UC-14: System Monitoring - UC-15: Admin Maintenance

---

## 7. Future Enhancements

- Mobile applications (Android/iOS)
- AI-based personalized training plans
- Multiplayer learning modes
- Coach-student live sessions

---

## 7. Appendix

### 7.1 Technology Stack Summary

- Backend: Django, DRF
- Frontend: Django Templates, React

- Database: PostgreSQL
- Payments: Stripe
- Chess Engine: Stockfish

---

## 7.2 Revision History

| Version | Date | Description |
|---|---|---|
| 1.0 | Initial | Initial SRS Draft |

---

**End of Document**